

Baseline and Preprocessing

Post EDA, we observed that, a per road type modelling strategy was needed, i.e separate lightGBM models for highway, street and residential.

We started with a robust pipeline featuring a per-road-type modeling strategy (training separate LightGBM models for Highway, Street, and Residential).

- Spatial Decoding: Translated geohash strings into continuous lat and lon coordinates to give the trees a physical map.
- Cyclical Time: Encoded time_minutes using sine/cosine transformations so the model understood that 23:59 is adjacent to 00:01.
- Target Transformation: Applied a Yeo-Johnson power transform to the demand target to reduce extreme skewness and stabilize the RMSE loss function.
- OOF Encoding: Used 5-Fold Out-of-Fold target encoding on specific interactions like (geo5, hour) to capture neighborhood baselines without target leakage.

Baseline Score: ~89.20

T-24 hours LAG Trap

K fold encoding was too smooth, and since all time stamps in day 49 are present in day 49, thought providing look ups would help.

- The Experiment: Replaced statistical averages with strict dictionaries mapped directly from Day 48.
- The Result: A catastrophic drop to 81.2.

Day 48 and Day 49 did not share the same baseline traffic behavior (e.g., transitioning from a quiet Sunday to a massive Monday sale event).

Further attempting to map and find relationship across the exact same data points that only varied in the day itself made no difference, there was no relationship to be found.

Transition to Spatio-Temporal

To move forward from just time series modelling, we shifted to linking both, and finding relationships between time and space together.

- Macro-Resolution (geo4): Extracted the first 4 characters of the geohash to create 39km x 19km bounding boxes, capturing city-wide district patterns.
- Traffic Archetypes (K-Means): Pivoted Day 48's 24-hour curves and clustered the locations into 5 archetypes (e.g., Business Parks vs. Residential Sleepers).
- The Geohash Ring: Attempted to calculate the spillover demand of the 8 surrounding grids (though LightGBM's native lat/lon awareness mostly absorbed this).

The Result: 89.4861

Picking an Ensemble Set Up

Used XGBoost and CatBoost alongside LightGBM to explore our excellent features in different ways.

- Built a tri model ensemble, optimized blend weights using optuna.
 - Score dropped to 87.
-

Tools used

- Standard data manipulation (Pandas and Numpy)
- Machine Learning frameworks (LightGBM, XGBoost, CatBoost)
 - Note: We also tried to frame a lstm around this problem, no success.
- Scikit learn ecosystem (yeo-johnson transform, Kmeans clustering, Kfold)
- Spatial Engineering (Pygeohash and geohash2)
- Optuna for hyperparameter studies
- Matplotlib, seaborn for visualisation.